

6장 합성곱 신경망 II (1)

🕒 생성일 @2025년 4월 7일 오후 2:32

1 이미지 분류를 위한 신경망

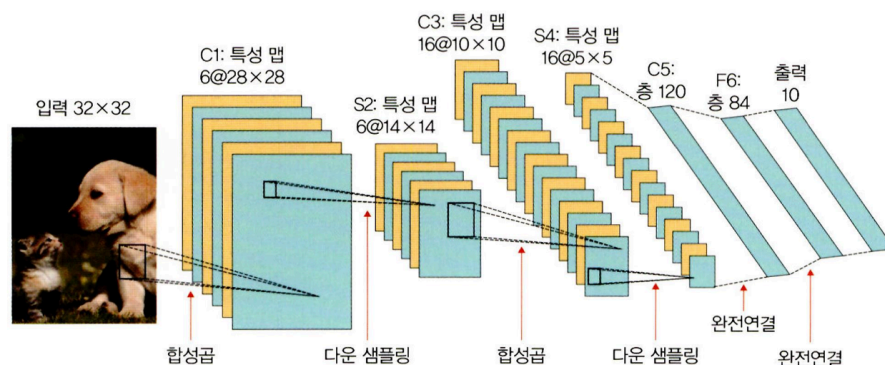
1. LeNet-5
2. AlexNet
3. VGGNet
4. GoogLeNet

1 이미지 분류를 위한 신경망

입력 데이터로 이미지를 사용한 분류 → 특정 대상의 영상 내 존재 여부 판단하는 것

1. LeNet-5

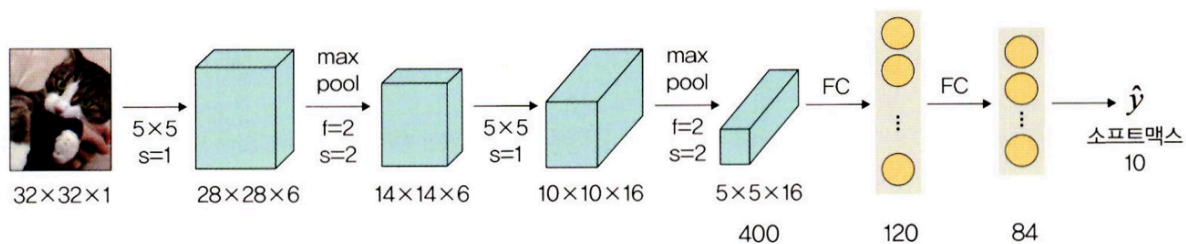
: 합성곱 (convolutional)과 다운 샘플링(sub-sampling)(or 풀링)을 반복적으로 거치면서 마지막에 완전 연결층에서 분류를 수행



LeNet-5 예시

- C_- : 합성곱층 / S_- : 풀링층 / F_- : 완전연결층
 1. $C1$ 에서 5 x 5 합성곱연산 → 28 x 28 크기의 특성맵 6개 생성
 2. $S2$ 에서 다운 샘플링 하여 특성맵 크기를 14x14로 축소
 3. $C3$ 에서 5x 5 합성곱연산 → 10x 10 크기의 특성맵 16개 생성하고
 4. $S4$ 에서 다운 샘플링 하여 특성맵 크기를 5x5로 축소
 5. $C5$ 에서 5x5 합성곱연산 → 1x1 크기의 특성맵 120개 생성
 6. $F6$ 에서 완전연결층으로 C5의 결과를 유닛(or 노드) 84개에 연결

[예제] 32 x 32 크기의 이미지에 합성곱층과 최대풀링층이 쌍으로 두 번 적용된 후 완전연결층을 거쳐
이미지가분류되는 신경망



계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

코드 6-1 필요한 라이브러리 호출

코드 6-2 이미지 데이터셋 전처리

1 torchvision.transforms

```
train_transforms = transforms.Compose(
    (a)
    [transforms.RandomResizedCrop(resize, scale=(0.5,1.0)),
     (b)
     transforms.RandomHorizontalFlip(),
     (c)
     transforms.ToTensor(), transforms.Normalize(mean, std)]
    (d)                      (e))
```

1. torchvision.transform : 이미지에 대한 전처리

- a. **Compose** : 이미지 변형 방식 묶음
- b. **RandomResizedcrop** : 입력 이미지를 주어진 크기로 조정,
 - **scale** : 원래 이미지를 임의의 크기만큼 면적을 무작위로 자르겠다는 의미
- c. **RandomtorizontalF1ip** : 주어진 확률로 이미지 수평 반전
 - 확률값 지정 X → 기본값 0.5의 확률로 이미지 수평 반전
= 훈련 이미지 중 반은 위아래 뒤집힌 상태 / 반은 그대로 사용
- d. **ToTensor** : 이미지 데이터 텐서로 변환
 - ImageFolder 메서드를 비롯, torchvision 메서드는 이미지 읽을 때 파이썬 이미지 라이브러리인 PIL 사용 → PIL 사용해서 이미지 읽으면 생성 되는 이미지는 범위가 [0, 255], 배열의 차원이(높이H x 너비W x 채널수C)로 표현
⇒ 효율적인 연산을 위해 torch.FloatTensor 배열로 변환 → 픽셀 값의 범위 [0.0, 1.0] 사이, 차원의 순서 (채널수C x 높이H x 너비W)
- e. **Normalize** : ImageNet 데이터의 각 채널별 평균과 표준편차에 맞는 정규화
 - 전이학습 에서 사용하는 사전 훈련된 모델들 대개 ImageNet 데이터셋에서 훈련
⇒ 사전 훈련된 모델 사용 위해서는 정규화 필요
 - (mean: 0.485, 0.456, 0. 406), (std: 0. 29, 0. 24, 0. 225) : ImageNet 이미지들의 RGB 채널마다 평균과 표준편차
 - OpenCV 사용해 이미지 읽을 경우, RGB 이미지가 아닌 BGR 이미지
→ 채널순서 주의

2

__call__ 함수

```
def __call__(self, img, phase):  
    return self.data_transform[phase](img)
```

- 클래스 호출할 수 있도록 하는 매서드
- 인스턴트가 호출 되었을 때 실행됨 ↔ `__init__` : 인스턴스 초기화 위해 사용
- 클래스에 `__call__` 함수가 있을 경우, 클래스 자체 호출 시, `__call__` 함수의 `return` 값이 반환

코드 6-3 이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리

1 고양이 이미지 데이터 가져옴

```
cat_images_filepaths = sorted([os.path.join(cat_directory, f)
                                (a) (b)
                                for f in os.listdir(cat_directory)])
                                (c)
```

- a. `sorted` : 데이터를 정렬된 리스트로 만들어 반환
- b. `os.path.join` : 경로와 파일명을 결합 or 분할된 경로를 하나로 합칠 때 사용
 - 디렉토리 (../chap06/data/dogs- vs- cats/Cat/) + 파일명 (cat.0.jpg)
= ../chap06/data/dogs-vs-cats/Cat/cat.0.jpg
 - 경로를 하나로 합침

```
import os
list_path = ['C:\\', 'Temp', 'user']
folder_path = os.path.join(*list_path)
folder_path
```

→ 실행결과: C: \\Temp\\user'

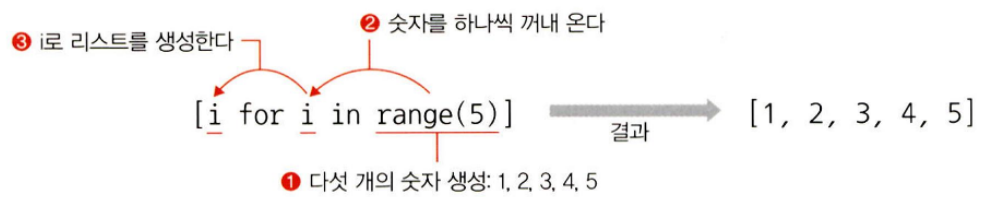
- c. `os.listdir` : 지정된 디렉토리 내 모든 파일 리스트 반환

2 image_filepaths에서 이미지 파일들 불러옴

```
correct_images_filepaths = [i for i in images_filepaths  
                             if cv2.imread(i) is not None]
```

(a) (b) (c)

a. for 반복문 → 가져온 데이터에 대해 i를 이용하여 리스트로 만들



b. 반복문(for) 이용 → images_filepaths에서 이미지 데이터 검색

c. 조건문(if) 의미, `CV2.imread()` 함수 이용 → 모든 이미지 데이터 읽어옴

3

Numpy.random.seed()

```
random.seed(42)
```

- 넘파이 random() 함수 : 임의의 난수 생성 → 난수를 생성하기 위해 사용되는 것이 시드값(seed value)
- Numpy.random.seed()메서드 : 상태 초기화 ⇒ 모듈 호출될 때마다 임의의 난수 재생성됨
- 특정 시드 값 부여 시 상태 저장 → 동일한 난수 생성
 - 시드값: 101 설정

```
import numpy as np

np.random.seed(101)
np.random.randint(low=1, high=10, size=10)
```

→ 결과: array ([2, 7, 8, 9, 5, 9, 6, 1, 6, 9])

- 시드값: 100 설정

```
np.random.seed(100)
np.random.randint(low=1, high=10, size=10)
```

→ 결과: array ([9, 9, 4, 8, 8, 1, 5, 3, 6, 3])

- 다시 시드값: 101 설정

```
np.random.seed(101)
np.random.randint(low=1, high=10, size=10)
```

→ 결과: array([2, 7, 8, 9, 5, 9, 6, 1, 6, 9])

⇒ 이전 시드값 101 설정했던 것과 같이 동일한 값 출력

코드 6-4 테스트 데이터셋 이미지 확인 함수

1 Cv2.cvtColor

```
cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

(a) (b)

1. `Cv2.cvtColor` : 이미지 색상 변경하기 위해 사용
 - a. 첫번째 파라미터 : 입력 이미지
 - b. 두번째 파라미터 : 변환할 이미지 색상 지정
→ BGR(Blue, Green, Red) 채널 이미지를 RGB(컬러)로 변경

2 이미지 전체 경로 정규화, 분할

```
true_label = os.path.normpath(image_filepath).split(os.sep)[-2]
```

(a) (b)

- a. `os.path.normpath` : 경로명 정규화
- b. `split(os.sep)` : 경로를 / 혹은 \ 를 기준으로 분할

3 predicted_label

```
predicted_label = predicted_labels[i] if predicted_labels else true_label
```

(a) (b)

- a. predicted_label 값 O → 해당 값을 predicted_label로 사용
- b. predicted_label 값 X → true_label 값을 predicted_label로 사용

코드 6-5 테스트 데이터셋 이미지를 출력

→ 기본적 데이터셋 준비 완료, 이후 학습 단계

1. 데이터셋에는 학습할 데이터의 경로 정의 → 경로에서 데이터 읽어옴

- 데이터셋 크기가 클 수 있으므로 `__init__` 에서 전체 데이터를 읽는 것 x 경로만 저장 → `__getitem__` 메서드에서 이미지 읽어옴
= 데이터를 어디서 가져올지 결정

2. 데이터 로더에서 데이터 셋의 데이터를 메모리로 불러옴

- 한꺼번에 전체 데이터 불러옴 x → 배치 크기만큼 분할하여 가져옴

코드 6-6 이미지 데이터셋 클래스 정의

- `DogvsCatDataset()` 클래스는 데이터 불러오는 방법을 정의
- 해당 예제의 목적: 다수의 개와 고양이 이미지가 포함된 데이터에서 이들을 예측하는 것
→ 레이블(정답) 이미지에서 고양이와 개가 포함될 확률을 코드로 구현
ex. 고양이가 있는 이미지의 레이블은 0, 개가 있는 이미지의 레이블은 1이 되도록 코드 구현

1 이미지 데이터에 대한 레이블값 가져옴

```
label = img_path.split('/')[-1].split('.')[0]
           (a)           (b)           (c)
```

- `img_path` : 이미지가 위치한 전체 경로 보여줌
→ 이때, 이미지 데이터의 레이블을 가져오려면 `/`, `.` 제거 필요 ⇒ b, c
- `img_path.split('/')[-1]` : 이미지 전체 경로에서 `/` 제거
- `split('.')[0]` : b 에서 얻은 결과에서 `.` 제거

코드 6-7 변수 값 정의

코드 6-8 이미지 데이터셋 정의

- 실행결과: 훈련데이터셋의 레이블이 1 값을 가짐 = '개'라는 이미지가 포함 / 0 값을 가짐 = '고양이'라는 이미지 포함 유추 가능

코드 6-9 데이터로더 정의

1

파이토치 데이터로더

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

(a) (b) (c)

1. 파이토치의 데이터로더는 배치 관리를 담당, 한 번에 모든 데이터 불러올 경우 메모리에 부담 → 데이터를 그룹으로 쪼개서 조금씩 불러옴
 - a. 첫 번째 파라미터: 데이터를 불러오기 위한 데이터 셋
 - b. `batch_size` : 한 번에 메모리로 불러올 데이터 크기
 - c. `shuffle` : 메모리로 데이터를 가져올 때 임의로 섞어서 가져오도록 함

코드 6-10 모델의 네트워크 클래스



Conv2d 계층에서의 출력크기 구하는 공식

$$\text{출력 크기} = (W - F + 2P) / S + 1$$

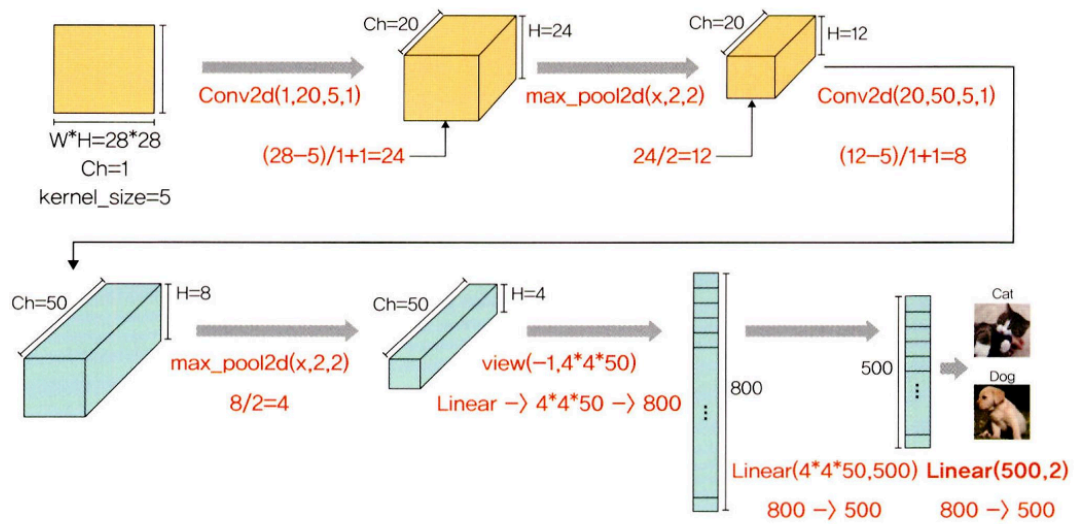
- W: 입력 데이터의 크기(input_volume_size)
- F: 커널 크기(kernel_size)
- P: 패딩 크기(padding_size)
- S: 스트라이드(strides)



MaxPool2d 계층에서의 출력크기 구하는 공식

$$\text{출력 크기} = IF / F$$

- IF: 입력 필터의 크기(input_filter_size) = Conv2d의 출력크기
- F: 커널 크기(kernel_size)



합성곱층과 풀링층의 크기(형태) 계산

코드 6-11 모델 객체 생성

코드 6-12 torchsummary 라이브러리를 이용한 모델의 네트워크 구조 확인

1 torchsummary

```
summary(model, input_size=(3,224,224))
```

(a) (b)

- a. 첫번째 파라미터: 모델의 네트워크
- b. 두번째 파라미터: 입력값으로 (채널(channel), 너비(width), 높이(height))가 됨
- torchsummary 라이브러리 이용 시 모델의 구조
 - 네트워크 내 파라미터 수와 구조가 이해하기 쉽게 표현
 - 출력 정보: 총 파라미터 수, 입력 크기, 네트워크의 총 크기 등

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216

ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0
=====		
Total params: 46,038,242		
Trainable params: 46,038,242		
Non-trainable params: 0		
=====		
Input size (MB): 0.57		
Forward/backward pass size (MB): 19.47		
Params size (MB): 175.62		
Estimated Total Size (MB): 195.67		
=====		

코드 6-13 학습가능한 파라미터 수 확인

→ 실행 결과: 많은 파라미터를 학습해야 하기 때문에 추후 데이터를 모델에 적용했을 때 긴 시간 소요 예상

⇒ 캐글에서 내려 받은 이미지 전체를 사용하는 것이 아닌 일부 이미지만 예제에서 사용

1 모멘텀 SGD

```
optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

(a) (b) (c)

1. 모멘텀 SGD : SGD에 관성이 추가된 것, 매번 기울기를 구하지만 가중치를 수정하기 전에 이전 수정방향(+, -)을 참고하여 같은 방향으로 일정한 비율만 수정되게 하는 방법
 - a. 첫번째 파라미터: 경사 하강법을 통해 궁극적으로 업데이트 하고자 하는 파라미터 ⇒ 가중치, 바이어스
 - b. lr (learning rate): 가중치 변경 시 얼마나 크게 변경할지 결정
 - c. momentum: SGD를 적절한 방향으로 가속화하며 흔들림(진동)을 줄여주는 매개변수

1 reduction 파라미터

```
criterion = nn.CrossEntropyLoss(reduction='mean')
```

- reduction 파라미터의 기본값은 'mean'
- 'mean'은 정답과 예측값의 오차를 구한 후 그 값들의 평균 반환
- 즉, 손실함수 특성상 전체 오차를 배치 크기로 나눔으로써 평균을 반환하기 때문에 epoch_loss를 계산하는 동안 loss.item()과 inputs.size(0)을 곱해줌

1

```
img = img.unsqueeze (0)
```

- `torch.unsqueeze` : 텐서에 차원을 추가할 때 사용 / `(0)` : 차원이 추가 될 위치
- ex. 형태가(3)인 텐서 → 0 위치에 차원을 추가 → 형태가 (1,3)이 됨 = 행 한 개와 열 세 개의 구조를 갖는 텐서
- ex. 2D에 텐서가 추가 될 때 ⇒ 형태가 (2,2)인 2D텐서
 - 0 위치에 차원을 추가 → 형태가 (1,2,2)가 됨 = 채널 1개, 행 2개, 열 2개
 - 1 위치에 차원을 추가하면 (2,1, 2)의 형태가 됨 = 채널 2개, 행 1개, 열 2개
 - 2 위치에 차원을 추가하면 텐서는(2,2, 1)의 형태가 됨 = 채널 2개와 행 2개, 열 1개

2

소프트 맥스 : 지정된 차원(dim)을 따라 텐서의 요소(텐서의 개별 값)가 (0,1) 범위에 있고 합계가 1이 되도록 크기 다시 조정

```
F.softmax(outputs, dim=1)[: , 1].tolist()
```

(a) (b) (c)

- outputs에 softmax를 적용하여 각 행의 합이 1이 되도록 함
- a의 값 중 모든 행(:)에서 두번째 칼럼을 가져옴

인덱스는 0부터 시작

인덱스	0	1	2	3	4
배열	10	11	12	13	14

- 배열을 리스트 형태로 변환

코드 6-19 테스트 데이터셋의 예측 결과 호출

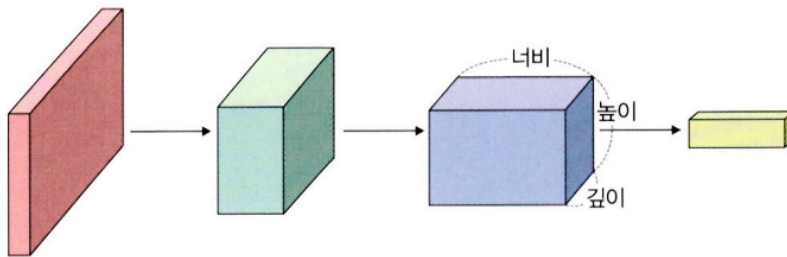
- 예측 결과 레이블이 0.5보다 크면 개를 의미, 0.5보다 작으면 고양이를 의미

코드 6-20 테스트 데이터셋 이미지를 출력하기위한 함수 정의

코드 6-21 테스트 데이터셋 예측 결과 이미지 출력

2. AlexNet

- CNN 구조

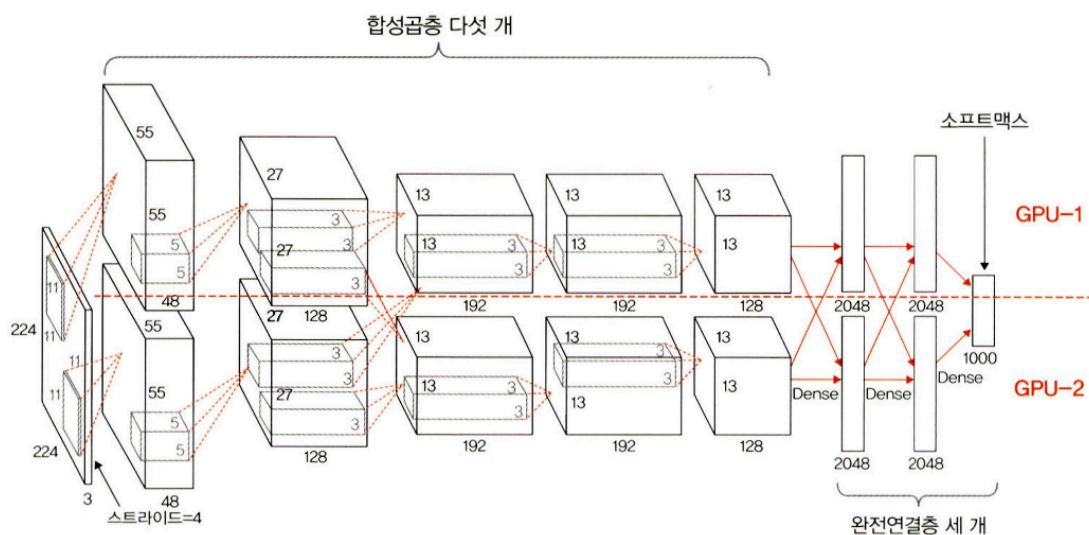


- CNN은 3차원 구조를 가짐
- 이미지 크기를 나타내는 너비(wich)와 높이(heigh) + 깊이(dcpth)
- 색상이 많은 이미지는 R/G/B 성분 세 개를 갖기 때문에 시작이 3
but, 합성곱을 거치면서 특성맵이 만들어지고 이것에 따라 중간 영상의 깊이가 달라짐

- AlexNet

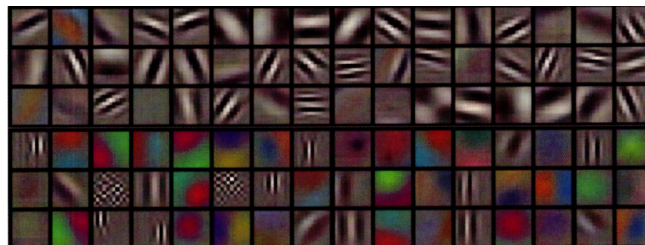
- 합성곱층 총 5개와 완전연결층 3개로 구성
- 맨 마지막 완전연결층은 카테고리 100개를 분류하기 위해 소프트맥스 활성화함수를 사용
- GPU 2개 기반 병렬구조

→ LeNet-5와 유사

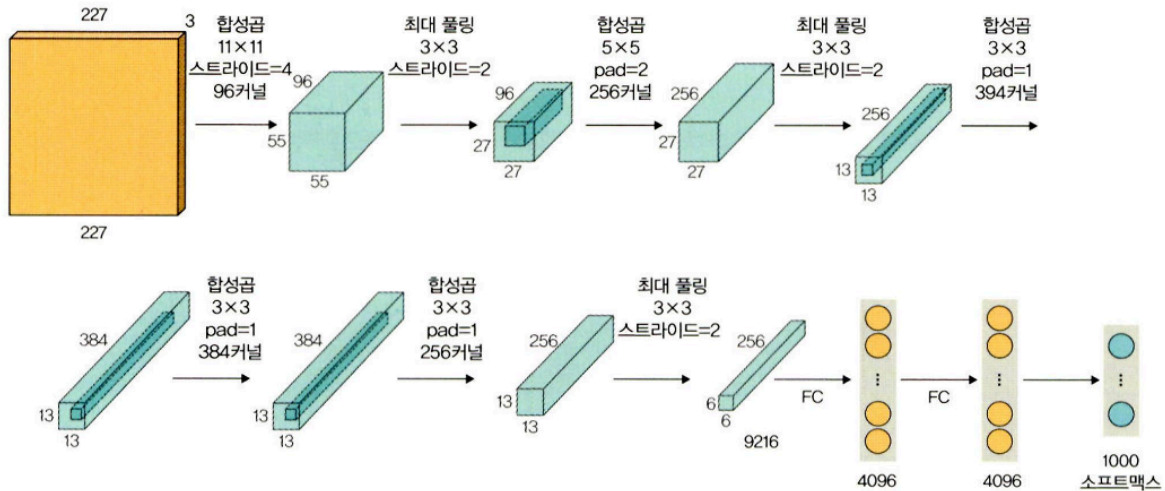


계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	–	–	–
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	–
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	–
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	–
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	1000	–	–	소프트맥스(softmax)

- AlexNet의 첫번째 합성곱층 커널의 크기: 11X11X3, 스트라이드 = 4, 특성맵 96개 생성 → 55x55x96 출력
- 첫번째 계층을 거치면서 GPU-1에서는 주로 컬러와 상관없는 정보를 추출하기 위한 커널이 학습, GPU-2에서는 주로 컬러와 관련된 정보를 추출하기위한 커널이 학습
- AlexNet GPU- 1 · 2 적용결과



코드 6-22 필요한 라이브러리 호출



코드 6-23 데이터 전처리

코드 6-24 데이터를 가져와서 훈련, 검증, 테스트 용도로 분류

코드 6-25 커스텀 데이터셋 정의

코드 6-26 변수에 대한 값 정의

코드 6-27 훈련, 검증, 테스트 데이터셋 정의

- 훈련 데이터셋의 크기 (3, 256, 256) → 해당 값은(채널, 너비, 높이)를 의미

코드 6-28 데이터셋을 메모리로 불러옴

- 합성곱(Conv2d) + (활성화함수(ReLU)+풀링(MaxPool2d)이 다섯번 반복 → 두 개의 완전연결층, 출력층으로 구성된 네트워크

코드 6-29 AlexNet 모델 네트워크 정의

1

`nn.ReLU(inplace=True)`

- `inplace` 연산: 연산에 대한 결과값을 새로운 변수에 저장하는 것이 아닌 기존 데이터를 대체하는 것
= 기존값을 연산결과값으로 대체함으로써 기존값들을 무시

2

```
self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
```

- `nn.AdaptiveAvgPool2d` : 풀링을 위해 사용
 - `AvgPool2d` : 풀링에 대한 커널 및 스트라이드 크기를 정의해야 동작
= (N, C, H.in, W.in) 크기의 입력을 (W, C, H.out, W.out) 크기로 출력하는 것

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times \text{padding}[0] - \text{kernel_size}[0]}{\text{stride}[0]} + 1 \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in} + 2 \times \text{padding}[1] - \text{kernel_size}[1]}{\text{stride}[1]} + 1 \right\rceil$$

- `AdaptiveAvgPool2d` : 풀링 작업이 끝날 때 필요한 출력 크기를 정의
 - 출력에 대한 크기만 지정 ↔ `AvgPool2d` : 커널크기, 스트라이드, 패딩 지정

⇒ 출력 크기에 대한 조정이 쉬워짐

⇒ 입력 크기에 변동이 있고 CNN 위쪽에 완전연결층을 사용하는 경우 유용

코드 6-30 model 객체생성

코드 6-31 옵티마이저 및 손실함수 정의

코드 6-32 모델 네트워크 구조 확인

코드 6-33 모델 학습함수 정의

코드 6-34 모델학습

코드 6-35 모델을 이용한 예측

코드 6-36 데이터 프레임 결과 확인

- label 값이 0.5보다 크면 개, 0.5 보다 작으면 고양이로 예측했음을 의미

코드 6-37 예측 결과를 시각적으로 표현하기 위한 함수정의

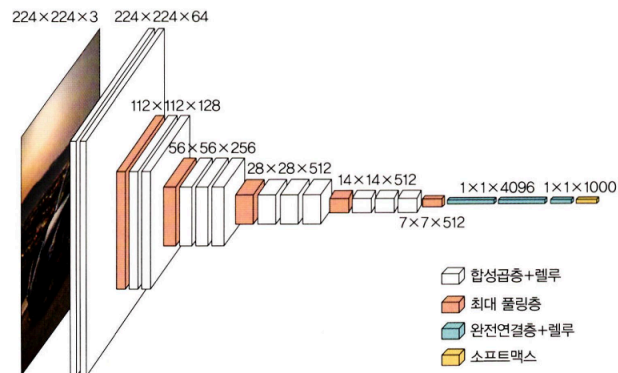
코드 6-38 예측 결과에 대해 이미지와 함께 출력

3. VGGNet

: 합성곱층의 파라미터수를 줄이고 훈련시간을 개선하려고 탄생

- 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 것이 VGG

- 깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3 x 3으로 고정
- 네트워크 계층의 총 개수에 따라 여러 유형의 VGGNetd이 있음 → VGG16, VGG19 등
 - VGG16 구조
 - 합성곱 커널의 크기 : 3X3 / 최대 풀링 커널의 크기 : 2x2 / 스트라이드 : 2
 - 64개의 224X224 특성맵(224X24X64)들이 생성됨
 - 마지막 16번째 계층을 제외하고는 모두 ReLU 활성화 함수가 적용됨



계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224x224	—	—	—
합성곱층	64	224x224	3x3	1	렐루(ReLU)
합성곱층	64	224x224	3x3	1	렐루(ReLU)
최대 풀링층	64	112x112	2x2	2	—
합성곱층	128	112x112	3x3	1	렐루(ReLU)
합성곱층	128	112x112	3x3	1	렐루(ReLU)
최대 풀링층	128	56x56	2x2	2	—
합성곱층	256	56x56	3x3	1	렐루(ReLU)
합성곱층	256	56x56	3x3	1	렐루(ReLU)
합성곱층	256	56x56	3x3	1	렐루(ReLU)
합성곱층	256	56x56	3x3	1	렐루(ReLU)
최대 풀링층	256	28x28	2x2	2	—
합성곱층	512	28x28	3x3	1	렐루(ReLU)
합성곱층	512	28x28	3x3	1	렐루(ReLU)
합성곱층	512	28x28	3x3	1	렐루(ReLU)
합성곱층	512	28x28	3x3	1	렐루(ReLU)
최대 풀링층	512	14x14	2x2	2	—

합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	–
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	1000	–	–	소프트맥스(softmax)

코드 6-39 필요한 라이브러리 호출

1

```
import copy
```

- 객체 복사를 위해 사용
 - 단순 객체 복사

```
original = [1,2,3] # original이라는 변수에 [1,2,3]저장
copy_o = original # copy_o에 original 복사 (shallow copy)
print(copy_o)
copy_o[2] = 10 # copy_o의 3을 10으로 값을 변경
print (copy_o)
print(original)
```

- [1, 2, 3]
[1, 2, 10]
[1, 2, 10]

→ copy_o 뿐만 아니라 원래값인 original의 3도 10으로 바뀜

- 얕은 복사 : `copy.copy()`

```
import copy
original = [[1, 2], 3]
copy_o = copy.copy(original) # original 값을 copy_o에 얕은 복사
print(copy_o)
copy_o[0][0] = 100 # copy_o의 [1,2]값을 100으로 변경
print(copy_o)
print(original)

append = copy.copy(original)
append[0].append(4) # 첫번째 리스트([1, 2])에 4를 추가
print(append)
print(original)
```

- [[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2, 4], 3]

→ copy_o에서 [1, 2] 값을 100으로 변경했더니 copy_o만 바뀜. 또한, [1,2]어를 추가했더니 original과 copy_o 모두 반영

- 깊은 복사 : `copy.deepcopy()`

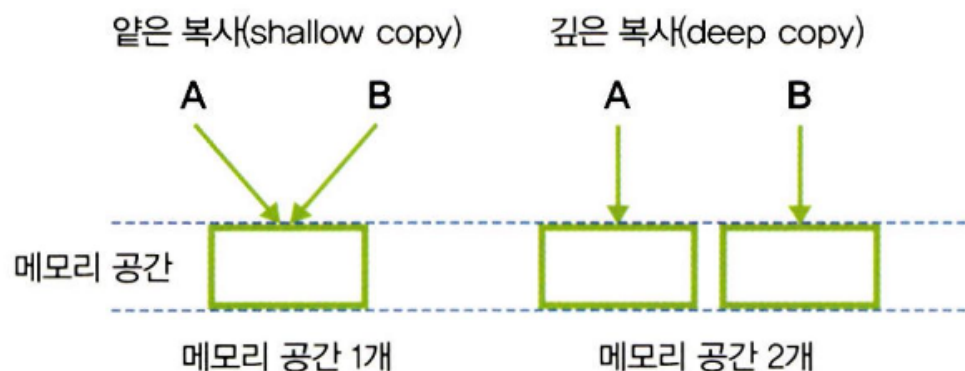
```
import copy

original = ([1, 2], 3]
copy_o = copy. deepcopy(original) # original 값을 copy_o에 깊은 복.
print (copy_o)
copy_o[0] = 100 # copy_o의 [1,2]값을 100으로 변경
print(copy_o)
print(original)

append = copy. deepcopy (original)
append[0]. append(4 # 첫번째 리스트 [1,2]에 4를 추가
print(append)
print(original)
```

- `[[1,2], 3]`
`[100, 3]`
`[[1, 2], 3]`
`[[1, 2, 4], 3]`
`[[1, 2], 3]`

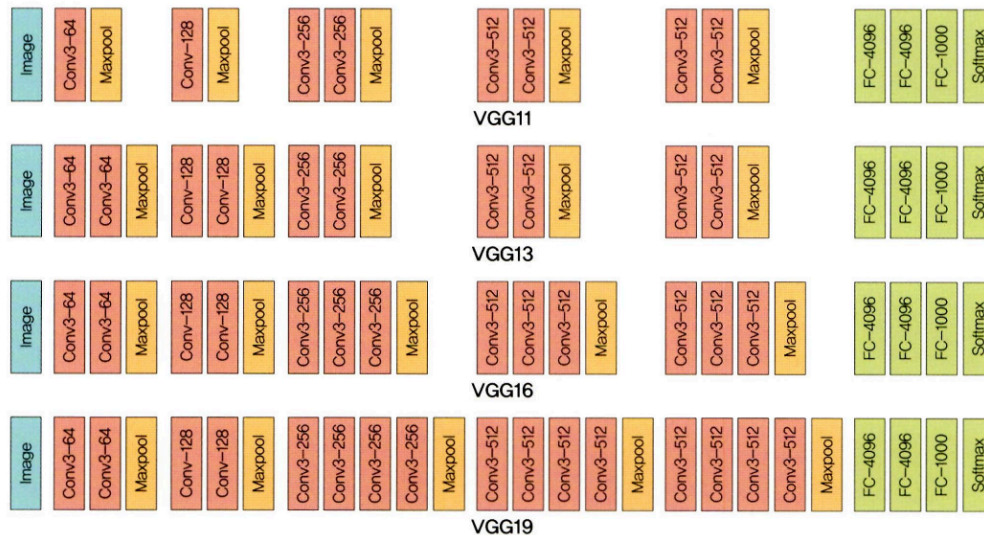
→ copy_o에서 [1, 2] 값을 100으로 변경했더니 copy_o만 바뀐 것은 동일. but [1,2]에 4를 추가했더니 copy_o 는 변경 되었으나 original은 그대로임



코드 6-40 VGG 모델정의

코드 6-41 모델 유형 정의

- VGG11, VGG13, VGG16, VGG19 모델의 계층을 정리
- 숫자(Output channel, 출력채널)는 Conv2d를 수행하라는 의미
- 출력채널(output channel)이 다음 계층의 입력채널(input channel)이 됨
- M은 최대풀링(maxpooling)을 수행하라는 의미



코드 6-42 VGG 계층 정의

1

```
assert c == 'M' or isinstance(c, int)
```

(a) (b)

- assert** : 가정설정문, 뒤의 조건이 True가 아니면 = `C == 'W'` 이 아니면 에러 발생
- isinstance** : 주어진 조건이 True인지 판단

```
print(isinstance(1, int)) # 1이 integer인지 판단
print(isinstance(1.2,int)) # 1.2가 integer인지 판단
print(isinstance('deep learning', str)) # dep learning이 string인지 판단
```

- True False True

코드 6-43 모델 계층 생성

1

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True)
```

- `batch_norm(BatchNormalization)` : 데이터의 평균을 0으로, 표준편차를 1로 분포시킴
 - 각 계층에서 입력 데이터의 분포는 앞계층에서 업데이트 된 가중치에 따라 변함
 - 즉, 각 계층마다 변화되는 분포는 학습속도를 늦출 뿐만 아니라 학습도 어렵게 함
 - 배치정규화: 각 계층의 입력에 대한 분산을 평균0, 표준편차 1로 분포 시키는 것

코드 6-44 VGG11 계층 확인

코드 6-45 VGG11 전체에 대한 네트워크

코드 6-46 VGG11 사전 훈련된 모델 사용

1

```
pretrained_model = models.vgg11_bn(pretrained=True)
```

(a) (b)

- a. `vgg11_bn` : VGG11 기본 모델에 배치 정규화가 적용된 모델을 사용하겠다는 의미
- b. `pretrained=True` : 사전 훈련된 모델을 사용하겠다는 의미

코드 6-47 이미지 데이터 전처리

1

```
transforms.Compose([transforms.Resize((256, 256)), transforms.RandomRotation(5),])
```

(a) (b)

- a. `transforms.Resize` : 이미지를 주어진 크기로 재조정
- b. `transforms.RandomRotation` : 5도 이하로 이미지를 회전

코드 6-48 ImageFolder를 이용하여 데이터셋 불러오기

1 ImageFolder

- 계층적인 폴더 구조를 가지고있는 데이터셋을 불러올 때 사용

코드 6-49 훈련과 검증데이터 분할

1

```
train_data, valid_data = data.random_split(train_dataset,  
                                           ①  
                                           [n_train_examples, n_valid_examples])  
                                           ②
```

1. `random_split()` : 훈련과 검증 데이터셋 나누는 용도로 사용, 데이터가 데이터로더로 넘어간 후에는 분리 불가능하므로 데이터셋 단계에서 진행
 - a. 첫번째 파라미터: 분할에 사용될 데이터셋
 - b. 두번째 파라미터: 훈련과 검증 데이터셋의 크기 지정 [훈련 데이터셋 크기, 검증 데이터셋 크기]

코드 6-50 검증 데이터 전처리

코드 6-51 훈련, 검증, 테스트 데이터셋 수 확인

코드 6-52 메모리로 데이터 불러오기

코드 6-53 옵티마이저와 손실함수 정의

코드 6-54 모델정확도 측정함수

1

예측과 정답이 일치하는 경우, 그 개수의 합을 correct 변수에 저장

```
correct = top_pred.eq(y.view_as(top_pred)).sum()
```

(a) (b) (c)

- a. `eq` : equal의 약자, 서로 같은지를 비교하는 표현식
- b. `view_as(other)` : other의 텐서 크기를 사용하겠다는 의미 = `view(other.size())`
 → `y.view_as(top_pred)` : y에 대한 텐서 크기를 top_pred의 텐서 크기로 변경한다는 의미
- c. 합계를 구하는 것, 예측과 정답이 일치하는 것들의 개수를 합산하겠다는 의미

코드 6-55 모델학습 함수정의

코드 6-56 모델성능 측정함수

코드 6-57 학습시간 측정함수

코드 6-58 모델학습

코드 6-59 테스트 데이터셋을 이용한 모델 성능측정

코드 6-60 테스트 데이터셋을 이용한 모델의 예측확인함수

1

`argmax` : 배열에서 가장 큰 값의 인덱스를 찾을 때 사용

```
top_pred = y_prob.argmax(1, keepdim=True)
```

(a) (b)

- a. 첫번째 파라미터: 행(axis=0) 또는 열(axis=1)을 따라 가장 큰 값의 색인을 찾음.
 → 1 : 열을 따라 가장 큰 값의 색인을 찾겠다는 의미
- b. `keepdim=True` : 출력 텐서를 입력과 동일한 크기로 유지하겠다는 의미

2

torch.cat : 텐서를 연결할 때 사용

	Col1	Col2	Col3
Row1			
Row2			
Row3			

axis=0(dim=0)

axis=1(dim=1)

- dim=0 : 행 / dim=1 : 열
- 예시 코드

```
import torch
x = torch.Tensor([[1, 2, 3], [2, 3, 4]])
y = torch.Tensor([[4, 5, 6], [5, 6, 7]])

print(torch.cat([x], dim=0)) ----- 행을 기준(dim=0)으로 x를 이어 붙입니다.
print('-----')
print(torch.cat([x, y])) ----- 단순히 x와 y를 결합하라는 의미이며 출력 결과는 (4×3) 형태가 됩니다.
print('-----')
print(torch.cat([x, y], dim=0)) ----- 행을 기준(dim=0)으로 x와 y를 이어 붙이면 (4×3) 텐서가 됩니다.
print('-----')
print(torch.cat([x, y], dim=1)) ----- 열을 기준(dim=1)으로 x와 y를 이어 붙이면 (2×6) 텐서가 됩니다.
```

```
tensor([[1., 2., 3.],
        [2., 3., 4.]])
-----
tensor([[1., 2., 3.],
        [2., 3., 4.]])
```

```
-----
[4., 5., 6.],
[5., 6., 7.]]
-----
tensor([[1., 2., 3.],
        [2., 3., 4.],
        [4., 5., 6.],
        [5., 6., 7.]])
-----
tensor([[1., 2., 3., 4., 5., 6.],
        [2., 3., 4., 5., 6., 7.]])
```

코드 6-61 예측 중에서 정확하게 예측한 것을 추출

1**max** : 최댓값 반환 / **argmax** : 최댓값을 갖는 인덱스 반환

```
x= torch.rand (5,3) # (5×3)형태의텐서생성

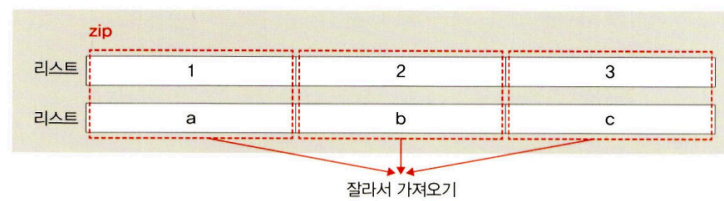
'''
tensor ([[0.6501, 0.6177, 0.5627],
 [0.3163, 0.5619, 0.6364],
 [0.8501, 0.9622, 0. 4792] ,
 [0.1859 , 0.6510 , 0.1824] ,
 [0.4304, 0.1170, 0.804911]])
'''

print(torch.max(x)) # max의 반환값 알아보기
print(torch.argmax(x)) # argmax의 반환값 알아보기

#tensor (0.9622)
#tensor (7)
```


2

`zip()` : 여러개의 리스트(or튜플)를 합쳐서 새로운 튜플타입으로 반환



```
a=[1, 2,3]
```

```
b=['a', 'b', 'c']
```

```
for x, y in zip(a, b): # a, b가 리스트로 구성되었기 때문에 for문을 통해 반복하여 실행
    print(x, y)
```

```
'''
```

```
1a
```

```
2b
```

```
3c
```

```
'''
```

3

`sort()` : 데이터 정렬

```
correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)
```

(a) (b)

- a. `reverse` : 내림차순으로 정렬
- b. `key` : 데이터 정렬 시 key값을 가지고 정렬, 기본값은 오름차순
 - a. `lamda` : 일종의 함수, 일반적으로 함수는 `def 함수명()` 과 같이 사용 but 람다 함수 함수명 없이도 사용 가능

```
def 함수명(매개변수):
    return 반환될 결과값

lamda 매개변수 : 반환될 결과값
```

- 정의와 동시에 사용할 수 있지만 함수명이 없고, 저장된 변수가 없기 때문에 재사용 불가능

```
x = torch.randn([4, 4]) # (4×4)크기를 갖는 임의의 텐서생성
print (x)
```

```
'''
```

```
tensor ( [[-0.3027, -0.3185, 0.4117, 0.4915],
          [3.112 ,1.3952, 0 .2522, - 0.5341],
          [-0.5381,-0.2026, -1.0494,-1.2712],
          [ -0.3689,1.8523 ,2.0229 ,0.57251]])
```

```
'''
```

```
max_elements, max_idx = torch.max(x, dim=0) # torch.max값을 가
print(max_elements)
print(max_idx)
```

```
'''
```

```
tensor([-0.3027, 1.3952, 2.0229, 0.5725])
tensor ([0, 1, 3, 3])
```

```
'''
```

1

`torch.add` : 더하라는 메서드

- `torch.add` ↔ `torch.add_`

```
x= torch. tensor ([1, 2])
y= .x add (10) # torch.add 적용

print (y)
print(x is y)

y = x. add_(10) # torch.add_적용
print (y)
print(x is y)

'''
tensor ([11, 12])
False

tensor ([11, 12])
True
'''
```

- `torch.add` 의 결과는 새로운 메모리 공간이 할당되어 저장
→ x와 y가 같은지 물었을 때 False라는 결과 출력
- `torch.add_` 는 새로운 공간 할당 없이 기존의 메모리에 위치한 값을 대체

1

`image.permute` : 축을 변경할 때 사용

```

x= torch.tensor([[1, 2, 3], [4, 5, 6]])
print(x)
x.permute (1, 0) # 차원(축) 0과 1을 바꿈

'''
tensor ([[1, 2, 3], [4, 5, 6]])
tensor ([[1, 4], [2, 5],[3, 6]])
'''

```

코드 6-64 예측 결과 이미지 출력

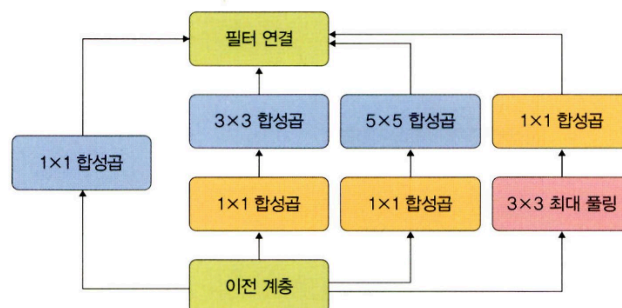
4. GoogLeNet

: 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습능력은 극대화 할 수 있는 깊고 넓은 신경망

- 인셉션(inception) 모듈
 - 특징을 효율적으로 추출하기 위해 1x1, 3x3, 5x5의 합성곱 연산 각각 수행
 - 3x3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 풀링연산에서는 드물게 패딩 추가 필요

⇒ 희소 연결

- 뻘뻘하게 연결된 신경망 대신 관련성이 높은 노드끼리만 연결하는 방법
 - ∴ CNN : 합성곱, 풀링, 완전연결층들이 서로 밀집(dense)하게 연결



- 인셉션 모듈의 네가지 연산
 - 1x1 합성곱

- 1X1합성곱+3x3합성곱
- 1X1합성곱+5x5합성곱
- 3×3최대풀링(maxpooling) + 1x1합성곱(convolutional)